1. RESUMEN OPERATIVO DEL PROYECTO

- **Objetivo del sistema a desarrollar**: Desarrollar un prototipo distribuido de gestión estratégica (Dune: Arrakis Dominion Distributed) donde el jugador controla una Casa Menor que gestiona enclaves de explotación, investigación y exhibición de criaturas en el planeta Arrakis.
- **Componentes obligatorios**:
  - Cliente de administración (Centro de mando).
  - Servicio de simulación (Gestión de rondas).
  - Servicio de persistencia (Carga y guardado).
  - Modelo compartido de dominio (Biblioteca de clases).
  - Registro de eventos (Historial y auditoría).
- **Requisitos técnicos clave**:
  - **Arquitectura distribuida**: Separación de responsabilidades en capas lógicas y despliegue físico por procesos o nodos independientes.
  - **Comunicación entre componentes**: Interacciones síncronas y asíncronas mediante protocolos como HTTP, AMQP o TCP.
  - **Persistencia**: Almacenamiento persistente en formato JSON.
  - **Concurrencia**: Gestión de hilos, procesos y exclusión mutua para evitar condiciones de carrera en el acceso a recursos compartidos.
  - **Gestión de errores**: Controles de integridad, gestión de excepciones, políticas de reintento y validación de mensajes.
  - **Seguridad básica**: Validación de entradas, protección de canales de comunicación y ficheros, y trazabilidad de fallos.
- **Restricciones del proyecto**: Implementación en C# sobre .NET, uso obligatorio de repositorio privado en GitHub con commits frecuentes, y defensa oral del sistema.

2. LISTA COMPLETA DE TAREAS (DESGLOSE TOTAL)

- **2.1. Diseño y arquitectura**:
  - Modelado de dominio: Definición de entidades (Partida, Enclave, Criatura, Instalación), DTOs para transferencia de datos y enumerados (Nivel adquisitivo, Medio, Alimentación).
  - Definición de proyectos: Creación de la solución con proyectos independientes para servicios, cliente y biblioteca compartida.
  - Decisiones de comunicación: Especificar el tipo de comunicación (RPC, RMI o mensajes) y protocolos.
- **2.2. Implementación técnica**:
  - Cliente de administración: Desarrollo de la interfaz para configurar la partida, consultar estado y lanzar operaciones.
  - Servicio de simulación: Implementación de la lógica para ejecutar rondas mensuales y recalcular estados del sistema.
  - Servicio de persistencia: Desarrollo del módulo encargado de la serialización y almacenamiento de datos.
  - Lógica de negocio:
    - Cálculo de visitantes (llegada y abandono) basado en salud de criaturas y hectáreas.
    - Sistema de alimentación, salud y metabolismo de criaturas (apetito base, edad adulta, ingesta).

        - Gestión económica: Solaris, costes de construcción, traslados y donaciones por criaturas favoritas.
        - Reproducción/clonación controlada (20% de éxito mensual) en instalaciones de aclimatación.
    - Sistema de eventos: Implementación de bitácora estructurada para auditoría técnica y funcional.
- **2.3. Sistemas distribuidos**:
    - Comunicación: Configuración de infraestructura síncrona/asíncrona y desacoplamiento de servicios mediante eventos.
    - Serialización: Implementación de mecanismos de transformación de objetos para el transporte entre nodos.
    - Manejo de fallos: Programación de políticas de reintento ante indisponibilidad de servicios y validación de mensajes entrantes.
    - Concurrencia y sincronización: Implementación de cerrojos, semáforos y estrategias de consistencia eventual entre servicios.
- **2.4. Persistencia**:
    - Guardado/carga: Funcionalidad de guardado bajo petición y automático al salir.
    - Formato: Uso de JSON interoperable para serialización de estado.
    - Consistencia: Asegurar que los componentes observen un estado compatible del sistema mediante la propagación de eventos.
- **2.5. Interfaz**:
    - Funcionalidades mínimas: Visualización de instalaciones, fondos, visitantes, eventos cronológicos y criaturas ordenadas por salud.
    - Opcional: Interfaz gráfica (WinForms, WPF) para obtención de puntos extra.
- **2.6. Testing y validación**:
    - Casos críticos: Validación de compras sin saldo, traslados inválidos y estados corruptos de carga.
    - Manejo de errores: Verificación de la trazabilidad de fallos de red y errores de serialización.
- **2.7. Memoria técnica**:
    - Apartados obligatorios: Análisis, fundamentos teóricos aplicados, diseño de arquitectura, concurrencia, persistencia, seguridad, estructuras de datos, ordenación/búsqueda, diagramas UML y fallos conocidos.
- **2.8. Entregas (hitos)**:
    - Entrega 1: Modelo de dominio, estructura de la solución, DTOs y justificación de arquitectura distribuida.
    - Entrega 2: Sistema ejecutable con persistencia JSON, cliente básico y comunicación operativa.
    - Entrega 3: Lógica de simulación completa, gestión de criaturas/visitantes, concurrencia y registro de eventos.
    - Entrega final: Solución completa compilable, memoria técnica y preparación para la defensa.

3. DIVISIÓN DEL TRABAJO EN 4 PERSONAS

- **Persona 1 (TX)**:

    - **Responsabilidades**: Diseño de la arquitectura distribuida, implementación de la comunicación entre procesos (servicios/cliente) y gestión de la concurrencia (threads, locks).
    - **Entregables**: Middleware de comunicación, biblioteca compartida de DTOs y contratos, y lógica de sincronización de estado.
- **Persona 2 (TX)**:
    - **Responsabilidades**: Lógica del servicio de simulación, motor de rondas, algoritmos de metabolismo/salud de criaturas y servicio de persistencia JSON.
    - **Entregables**: Módulo de simulación funcional, fórmulas de cálculo (visitantes/economía) y sistema de carga/guardado de ficheros.
- **Persona 3 (NO-TX)**:
    - **Responsabilidades**: Redacción de la memoria técnica (análisis, fundamentos, justificación técnica), diseño de diagramas UML y documentación de estructuras de datos.
    - **Entregables**: Documento de memoria técnica completo y diagramas estructurales del sistema.
- **Persona 4 (NO-TX)**:
    - **Responsabilidades**: QA y validación de errores, diseño de la interfaz de usuario (Centro de mando), implementación del registro de eventos y auditoría.
    - **Entregables**: Interfaz de administración operativa, plan de pruebas con casos críticos validados y logger de eventos integrado.
- **Dependencias**: Las Personas TX deben definir los contratos (Persona 1) y la lógica (Persona 2) para que las Personas NO-TX puedan documentarlos (Persona 3) y validarlos mediante pruebas (Persona 4).

4. PLAN TEMPORAL OPTIMIZADO

- **Fase 1 (Inicio - Hito 1)**: Tareas críticas de modelado de dominio y diseño de proyectos. Definición de contratos de comunicación (DTOs) que desbloquean el desarrollo paralelo de servicios.
- **Fase 2 (Desarrollo - Hito 2)**: Implementación de la infraestructura de red (comunicación cliente-servidor) y el servicio de persistencia básico. Permite el flujo de datos inicial.
- **Fase 3 (Integración - Hito 3)**: Desarrollo del núcleo de simulación (rondas, criaturas) y aplicación de mecanismos de sincronización/concurrencia. Integración del sistema de eventos para trazabilidad.
- **Fase 4 (Cierre - Entrega Final)**: Refinamiento de la interfaz de usuario, ejecución de planes de pruebas exhaustivos de errores y redacción final de la memoria técnica justificativa.
- **Orden lógico**: El diseño del modelo (Hito 1) desbloquea la persistencia y comunicación (Hito 2); estas permiten la implementación de la lógica de simulación compleja (Hito 3), finalizando con la documentación y pulido (Hito 4).